

(a) What I built. A short description of the algorithm or data structure you implemented and the key design decisions you made: language, data representation, any simplifications or extensions relative to the textbook version, and anything a reader needs to know to navigate the code. This should orient a reader before they open the repository, not duplicate the README

3 main sections of the codebase:

- Main code containing the CLI interface code
- AStar file containing the AStar search algorithm
 - Node.h containing node structure for AStar
- Maze file containing maze generation code

I built a maze puzzle game where the program can generate a maze using a 20 by 20 grid of nodes which can either be walls or open paths. The maze is generated using a randomised depth first search (DFS) backtracking algorithm to create paths for the maze. The main algorithm for this maze puzzle game is the use of AStar search to find the most efficient solution to the generated maze. The CLI allows the user to generate new mazes, place the goal node on any open path node and then show the solution using the AStar search algorithm.

In the program the maze is represented as a 2d vector to store the grid where walls were represented as a 1 and open path nodes were represented with a 0. Each node uses the node structure in Node.h where it includes the position of the node, cost variables for the AStar algorithm and the parent node to store the last position of the path. In order to move through the maze, movements are only in 4 directions, up, down, left and right.

(b) The track-specific writeup.

Track B — the tool: what it does, how the algorithm sits inside it, the interface decisions you made, and a worked example of it in use;

Maze solving game where the user can generate a maze, select a goal node and then attempt to solve the maze on their own to find the most efficient path, then they can use the AStar algorithm to solve the maze finding the most efficient path possible and check if they also found the best path to the goal node.

The AStar search algorithm sits in the AStar.cpp file where the aStar function parameters take in a 2D vector for the maze, and 2 nodes, the starting node and the goal node. It then uses a priority queue prioritising the lowest f value (estimated total path cost). It then searches through the 4 possible directions and uses the recorded g value from the current node to compare to the g value of reaching the new node through the current node. If the g value of the new node is smaller than the new g value coming from the current node, then the node's cost and parents are updated. The neighbour node is then created with this new information and its parent and pushed into the priority queue. This is looped until the algorithm reaches the goal node, where the algorithm then uses the parent nodes to build the path backwards until it reaches the starting node and the reverse of the path vector is returned.

In the CLI the user is prompted with 4 options: Generate maze, select goal, solve the maze with the AStar algorithm, exit. Selecting Generate maze, generates a new maze which is displayed whenever the user chooses a new option. Select goal prompts the user to select the row and column that the user wants the goal node to be placed and is shown on the maze. Solving the maze using AStar search will update the maze and show the best path to the goal node using “-” to denote the path. The final option to exit the game closes the interface. The main design choice I made was displaying the maze every time the user chooses an option. This way the user can see whenever an update happens to the maze and can intuitively understand how their options affect the maze.

Initially, no maze is displayed since the user needs to generate the maze before using any other option. If other options are chosen, there will be error messages letting the user know they need to generate the maze first. The user selects the first option to generate the maze and once the maze is generated it will be displayed at all times. The user then picks the second option to give the coordinates of where to place the goal node using the row and column. The maze is then updated to show the goal node where the user can attempt to solve the maze on their own. The third option to solve the maze with AStar search then updates the maze display with the shortest path to the goal node. The user can regenerate a new maze as many times as they want and needs to select the goal node each time before showing the solution. Choosing the 4th option just exits the program once the user is finished.

(c) What I learned. What surprised you? What did you initially get wrong? What is something you understand now that you didn't before starting? Specific examples carry far more weight here than general statements. A sentence like "I was surprised that the Fibonacci heap was slower than std::priority_queue on every workload I tried, and I now believe this is because of cache behaviour" is the kind of content I'm looking for.

One of the most interesting parts of my program that I had to change once I got everything working together was actually the maze generation. Originally starting with all open nodes, I used a random 30% chance that a node will either be a wall to generate the maze. This caused some nodes to be unreachable with a path from the start and also generated mazes that didn't look like mazes. The mazes generated like this had large open areas or large areas filled with walls. I wanted to make the maze actually look more like a realistic maze with any open node being accessible from the starting node. This type of maze is actually called a connected maze (DoYouMaze, 2026). I found that starting with a grid where all the nodes are walls and then using a randomised depth first search (DFS) using backtracking to carve out open nodes creates paths until there is nowhere else to go for a more realistic maze with long paths (Oliver Giulieri, 2026). Initially, the completely randomly generated maze used a 10x10 grid of nodes. However, after implementing the DFS maze generation the maze was too large to be displayed on a 10x10 grid for it to have meaningfully different paths to follow. I decided that changing it to 20x20 was more applicable for a connected maze rather than having it be 1x10 which was too easy to complete.

AI Use:

- I used ChatGPT to generate blocks of code which I put together manually into my codebase
- Did not generate full files or big chunks of code
- Only copied small chunks of code, no copy and paste and commented every line of code for understanding
- Only once comments were written did I continue with the next step of my implementation
- Some lines were left out by ChatGPT one notable instance was that I prompted it to compare its code to my own and it missed a line where it was meant to have `#include "Node.h"` at the top but it wasn't there in the ChatGPT code
- ChatGPT started getting confused with lines of code where I initialised the start and end nodes to be non wall nodes however, when transitioning to a user selected finishing node, I deleted the line where it always made the end goal a non wall but the LLM thought I kept it and mistakenly told me to delete the line where I make the start node a non wall.
- Created an option menu and ChatGPT recommended having a separate option for displaying the maze but it was intuitively not good so I redid additional code adding code to display the maze every time an option is chosen so that the user can see the maze and see updates to the maze every time an option is picked
- Originally the code for displaying the maze and menu for my maze puzzle would display the default goal node when printing the full maze, however I added the extra parameter of `goalSelected` into my `displayMaze` function so I could pass through if the goal had been selected or not. Then I used the boolean variable in the `elseif` statement so if the goal node hasn't been selected then it won't print the default goal node.

References:

- Belwariar, R. (2026). A* Search Algorithms. GeeksforGeeks
<https://www.geeksforgeeks.org/dsa/a-search-algorithm/>
- DoYouMaze. (2026). How to Make a Connected Dots Maze. DoYouMaze.
<https://www.doyoumaze.com/blog/how-to-make-a-connected-dots-maze#DEF>
- Giulieri, O. (2026). About Mazes. Maze Maker.
<https://evoluteur.github.io/maze-maker/about.html>
- Kartik. (2026). Depth First Search or DFS for a Graph. GeeksforGeeks.
<https://www.geeksforgeeks.org/dsa/depth-first-search-or-dfs-for-a-graph/>
- Patel, A. (2026). Introduction to A*. Theory.Stanford.
<https://theory.stanford.edu/~amitp/GameProgramming/AStarComparison.html>
- Yan, Y. (2023). Research on the A Star Algorithm for Finding Shortest Path.
https://www.researchgate.net/publication/370573811_Research_on_the_A_Star_Algorithm_for_Finding_Shortest_Path